

30

LLM Inference & Deployment Interview Questions

The round that separates builders from API callers.

KV cache · continuous batching · quantization · GPU memory math · one question per page, full answers.

Naved Khan
Senior Gen-AI Engineer

READ THIS FIRST

Everyone can call a model. Serving one is the interview.

Product companies test inference because it can't be faked: memory math, latency budgets, and cost per million tokens either add up or they don't. These 30 answers add up.

“Why is LLM inference so expensive?”

— THE FULL ANSWER

- ▶ Every generated token requires a **full pass through billions of parameters** — and answers are hundreds of tokens.
- ▶ The hardware is expensive, and it's **memory-bound**: the model weights plus attention caches must live in scarce GPU memory.
- ▶ And unlike training, inference runs **forever**, on every request, at user-facing latency.

“What are prefill and decode, and why do they behave differently?”

— THE FULL ANSWER

- ▶ **Prefill**: the model reads your whole prompt in one parallel pass. Compute-heavy, but fast per token.
- ▶ **Decode**: generating the answer, strictly **one token at a time**. Each step re-reads cached state — memory-bandwidth-bound.
- ▶ This split explains everything downstream: time-to-first-token is **prefill**, tokens-per-second is **decode**.
- ▶ Naming the two phases unprompted is the fastest senior signal in this round.

“Explain the KV cache in plain words.”

— THE FULL ANSWER

- ▶ To generate a token, the model attends over everything before it. Recomputing that **every step** would be brutal.
- ▶ So the attention state of processed tokens is **cached** — keys and values, hence KV.
- ▶ The cost: that cache **eats GPU memory per token of context**, which is why long contexts limit how many users fit on one GPU.
- ▶ One line: the KV cache trades **memory for speed**, and serving economics is managing that trade.

“Which metric matters: time-to-first-token or tokens-per-second?”

— THE FULL ANSWER

- ▶ Both, for different people. **TTFT is the user's metric** — under a second, streamed, or the product feels dead.
- ▶ **Throughput is the CFO's metric**: total tokens per GPU-hour decides cost.
- ▶ They fight: bigger batches raise throughput and hurt tail latency.
- ▶ The answer that wins: "I set a TTFT budget first, then **maximize throughput inside it.**"

“Do the GPU memory math for an 8B model.”

— THE FULL ANSWER

- ▶ Weights: parameters × bytes. At fp16, **8B × 2 bytes = 16 GB** before anything else.
- ▶ Add the KV cache — it grows with **context length × concurrent users** — plus activation overhead.
- ▶ That's why an 8B model doesn't fit comfortably on a 24 GB card at real concurrency, and why **quantization exists**.
- ▶ Doing this math out loud turns "we'll self-host" into a capacity plan.

```
# fp16 weights
```

```
8e9 params × 2 bytes = 16 GB
```

```
# int8 → 8 GB · 4-bit → ~4 GB, + KV cache on top
```

“A user says the model “types slowly.” What's actually slow?”

— THE FULL ANSWER

- ▶ Translate the complaint: slow first word = **prefill or queueing**. Slow streaming after = **decode throughput**.
- ▶ Queueing is the usual villain at load: the request waited for a batch slot before prefill even started.
- ▶ So instrument the three separately — **queue time, TTFT, tokens/sec** — or you'll fix the wrong thing.
- ▶ “It's slow” is three different bugs wearing one costume.

“What does quantization actually do, and what does it cost?”

— THE FULL ANSWER

- ▶ It stores weights in **fewer bits** — $\text{fp16} \rightarrow \text{int8} \rightarrow 4\text{-bit}$ — shrinking memory 2–4× and speeding up memory-bound decode.
- ▶ The cost is a small accuracy loss that you **measure, not assume**: run the golden set on both and compare.
- ▶ Modern 4-bit methods keep most quality for most tasks, which is why they're the default for self-hosting.
- ▶ The rule: quantize until your **eval** objects, not until the benchmark does.

“What is continuous batching, and why did it change everything?”

— THE FULL ANSWER

- ▶ Old batching waited for a group of requests, ran them **in lockstep**, and the longest answer held everyone hostage.
- ▶ **Continuous batching** admits and retires requests every step: a finished request's slot is refilled immediately.
- ▶ Result: 2–10× throughput at the same hardware, and it's why engines like **vLLM** became the standard.
- ▶ It's the single most important serving optimization of the last few years — say it with that weight.

“What problem does PagedAttention solve?”

— THE FULL ANSWER

- ▶ KV caches used to need **contiguous** GPU memory — reserved at max length, mostly wasted.
- ▶ PagedAttention manages the cache like **virtual memory**: fixed-size blocks, allocated as needed, near-zero fragmentation.
- ▶ That reclaimed memory becomes **more concurrent users per GPU** — it's the engine under vLLM's throughput.
- ▶ Plain words: it did for KV caches what paging did for RAM in the 1970s.

```
vllm serve meta-llama/Llama-3.1-8B-Instruct \\  
--max-model-len 8192 --gpu-memory-utilization 0.90
```

“Explain speculative decoding.”

— THE FULL ANSWER

- ▶ A small **draft model** guesses several tokens ahead; the big model **verifies them in one pass**.
- ▶ Guesses that match are accepted in bulk — output is **identical** to the big model's, just faster.
- ▶ Wins are workload-dependent: predictable text speeds up a lot, creative text less.
- ▶ The interview point: it trades **a little compute for a lot of latency**, with zero quality risk.

“Where does caching live on the serving side?”

— THE FULL ANSWER

- ▶ **Prefix caching**: shared prompt beginnings — system prompts, few-shot blocks — keep their KV cache across requests.
- ▶ **Semantic caching**: repeated questions skip the model entirely.
- ▶ Together they can cut real bills dramatically, because production traffic is **far more repetitive** than people expect.
- ▶ Design consequence you already know: stable prefix first. The server is why that rule pays.

“When does distillation enter the serving story?”

— THE FULL ANSWER

- ▶ When the cascade matures: prove the workload on a big model, then **train a small model on its outputs** for the routine traffic.
- ▶ A distilled 8B serving 70% of requests changes the economics completely — **10× cheaper per token**, faster, self-hostable.
- ▶ Gate it like everything else: the small model must **pass the same golden set**.
- ▶ This is where inference, cost, and fine-tuning meet — connecting them is the senior move.

“Self-host or use an API? Give me the decision rule.”

— THE FULL ANSWER

- ▶ **API by default:** zero infra, best models, elastic. Self-host when one of three flips: **data residency**, **unit economics at sustained volume**, or hard latency control.
- ▶ The break-even is a spreadsheet, not a vibe: GPU-hours + engineers vs. tokens × price.
- ▶ Most teams overestimate their volume and underestimate the **ops burden** — say that, it's the experienced take.
- ▶ And hybrid is normal: API for the frontier model, self-hosted small models for routine traffic.

“You're given one 80 GB GPU. What can you actually serve?”

— THE FULL ANSWER

- ▶ Work the math: a 70B model at fp16 needs ~140 GB — **doesn't fit**. At 4-bit, ~35 GB — fits with room for KV cache.
- ▶ An 8B at fp16 uses 16 GB, leaving ~60 GB of cache — that's **real concurrency** at long contexts.
- ▶ So the answer is a menu: big model quantized for quality, small model for throughput, or both with a router.
- ▶ The habit being tested: you **size before you promise**.

“How do you scale from one GPU to twenty?”

— THE FULL ANSWER

- ▶ Inference replicas are **stateless** — scale horizontally behind a router, session-affinity only for cache locality.
- ▶ Watch the real constraint: **GPU memory per replica** decides concurrency; the router balances by load, not round-robin.
- ▶ Autoscale on **queue depth and TTFT**, never CPU — GPUs saturate silently.
- ▶ Models too big for one card add tensor parallelism, at an interconnect cost you must name.

“How do you serve fifty fine-tuned variants without fifty GPUs?”

— THE FULL ANSWER

- ▶ **Multi-LoRA serving**: one shared base model in memory, adapters hot-swapped per request.
- ▶ Adapters are megabytes, not gigabytes — so one GPU pool serves **hundreds of customizations**.
- ▶ It's the pattern behind per-tenant and per-task fine-tunes at sane cost.
- ▶ One more reason PEFT won: the serving story, not just the training story.

“Design the streaming path from GPU to browser.”

— THE FULL ANSWER

- ▶ Tokens flow out as generated: engine → gateway → **SSE** to the client, rendered incrementally.
- ▶ Wire **cancellation end to end**: a closed tab must stop generation, or you pay for text nobody reads.
- ▶ Timeout on TTFT specifically — if the first token misses budget, fail over; don't leave a silent screen.
- ▶ Streaming isn't UX polish. It's the only thing that makes multi-second generations feel alive.

“The provider goes down at peak. What did you build for this day?”

— THE FULL ANSWER

- ▶ **Brownout, not blackout:** circuit breaker trips, traffic shifts to a fallback tier — smaller model, second provider, or cached answers.
- ▶ The UI degrades honestly: "answers may be slower right now."
- ▶ Failover is **rehearsed**, not documented: an unpracticed fallback is a rumor.
- ▶ Teams with no degraded mode have two modes: up, and apologizing.

“Decompose a 3-second response into a budget.”

— THE FULL ANSWER

- ▶ Queue **<100 ms**, retrieval ~200 ms, prefill to first token **<800 ms**, then streaming carries the rest.
- ▶ Each line is owned and measured separately — an SLA without a decomposition is a wish.
- ▶ The dominant term is decode, which is why perceived latency is managed with **streaming**, not raw speed.
- ▶ Present it as a table in interviews. Budgets impress; adjectives don't.

“P95 latency is fine, P99 is terrible. What's going on?”

— THE FULL ANSWER

- ▶ Tail latency in LLM serving is usually **queueing plus long-context outliers**: one 100k-token prompt stalls a batch slot.
- ▶ Fixes: **separate queues by prompt size**, cap max context per tier, admission control before saturation.
- ▶ Also look for cache-miss storms after deploys — cold prefix caches spike tails.
- ▶ The principle: tails are **scheduling problems** before they're model problems.

“How do you load-test an LLM service honestly?”

— THE FULL ANSWER

- ▶ **Recorded real traffic**, not synthetic uniform prompts — length distribution is everything in a memory-bound system.
- ▶ Ramp until TTFT breaks budget: that knee is your **true capacity**, not the GPU utilization number.
- ▶ Test the ugly mixes: long-context spikes, burst arrivals, cache-cold restarts.
- ▶ Report capacity as "X req/s at TTFT < 1s, P99 < 3s" — a number with conditions attached.

“Walk me through cost per million tokens, self-hosted.”

— THE FULL ANSWER

- ▶ GPU-hour price ÷ tokens generated per hour. An 80 GB card at \$2–4/hr pushing ~1–2k tokens/sec lands around **\$0.30–1.10 per million output tokens**.
- ▶ Then the honest additions: **utilization** (idle GPUs still bill), engineers, and failover headroom.
- ▶ Compare against API prices at your real volume — that's the build-vs-buy spreadsheet.
- ▶ Numbers are illustrative; the **method** is what the interviewer is buying.

“What do you monitor on an inference fleet?”

— THE FULL ANSWER

- ▶ The user-facing three: **queue time**, **TTFT**, **tokens/sec**, all at P95/P99.
- ▶ The capacity three: GPU memory headroom, **KV-cache utilization**, batch occupancy.
- ▶ And the money one: **tokens per GPU-hour**, trended weekly.
- ▶ Quality drift belongs to the eval suite — infra dashboards can't see a worse answer, only a slower one.

“How do rate limits and backpressure work here?”

— THE FULL ANSWER

- ▶ **Admission control at the gateway:** per-tenant token budgets, 429 with Retry-After, queues with a max depth.
- ▶ Shed load early and explicitly — a fast honest rejection beats a 40-second wait.
- ▶ Priority tiers matter: interactive traffic preempts batch jobs.
- ▶ The goal is a system that **degrades by policy**, not by whoever got lucky in the queue.

“Ship a new model version to the fleet without breaking anyone.”

— THE FULL ANSWER

- ▶ It's a deploy: **eval suite first**, then blue-green — new version on a slice, judge scores and latency compared, then cutover.
- ▶ Keep the old version warm for instant rollback; model behavior diffs are subtler than code diffs.
- ▶ Watch prompts too: a new model often wants **less scaffolding**, and your tuned prompts become dead weight.
- ▶ "A model swap is a deploy" — say the sentence; it summarizes the whole discipline.

“How do you capacity-plan for next quarter?”

— THE FULL ANSWER

- ▶ Forecast in **tokens**, not requests: tokens/user/day × users, split by cache-hit rate.
- ▶ Convert to GPU-hours via measured throughput, add failover headroom (~30%), decide reserved vs on-demand.
- ▶ Re-forecast monthly — AI usage curves bend fast in both directions.
- ▶ Capacity planning is the interview's hidden test of whether you've **paid a GPU bill**.

“When is self-hosting the wrong answer, even at scale?”

— THE FULL ANSWER

- ▶ When the frontier model's **quality gap is the product** — no ops savings buys back lost capability.
- ▶ When your team can't staff a 24/7 GPU fleet honestly.
- ▶ When volume is spiky: elastic APIs eat spikes; owned GPUs idle on them.
- ▶ Saying "we shouldn't self-host" with reasons is the same senior signal as every other "not yet" in this series.

“What's the single highest-leverage serving optimization today?”

— THE FULL ANSWER

- ▶ For most fleets: **continuous batching + prefix caching**, together — an engine default and a prompt-layout habit.
- ▶ They compound: batching fills the GPU, caching shrinks what each request costs to fill it with.
- ▶ Quantization is next when memory is the wall.
- ▶ The trap answer is "a faster GPU" — hardware last, after the software wins are banked.

“An agent workload just doubled your inference bill. Connect the layers.”

— THE FULL ANSWER

- ▶ Agents multiply **input tokens**: every step re-sends context, so the serving fix and the context fix are the same fix.
- ▶ Serving side: prefix caching makes re-sent context cheap. App side: compaction and fewer steps make it small.
- ▶ Route agent steps by difficulty — planning on the big model, tool-glue on a small one.
- ▶ This question tests whether your layers talk to each other. Answer across all of them.

“Close it out: your serving philosophy in one line.”

— THE FULL ANSWER

- ▶ **"Set the latency budget first, then buy throughput inside it — and measure in tokens, not vibes."**
- ▶ Behind it: TTFT is sacred, memory is the real currency, and every optimization passes the eval gate.
- ▶ Then one receipt: a number you moved — "tokens per GPU-hour up 3× after continuous batching and prefix cache."
- ▶ Philosophy plus a receipt. Same formula as every round in this series.

Preparing for AI interviews?



Naved Khan

Senior Gen-AI Engineer

Full deep-dive decks on **RAG, agents, evals, MCP, system design, safety, and context engineering** are on my profile. New guides weekly.